Introduction

This report summarizes a passion project started in December of 2023 which morphed into, and concluded in, a senior design project completed in the Winter semester of 2024. This synopsis is adapted from a more exhaustive report.

Design Goals

- 5 degree of freedom (DOF) robot arm, all joints revolute
- ≥700mm reach
- ≥100mm/s end effector speed
- 0.5kg payload
- BCP jogging
- TCP jogging using inverse kinematics

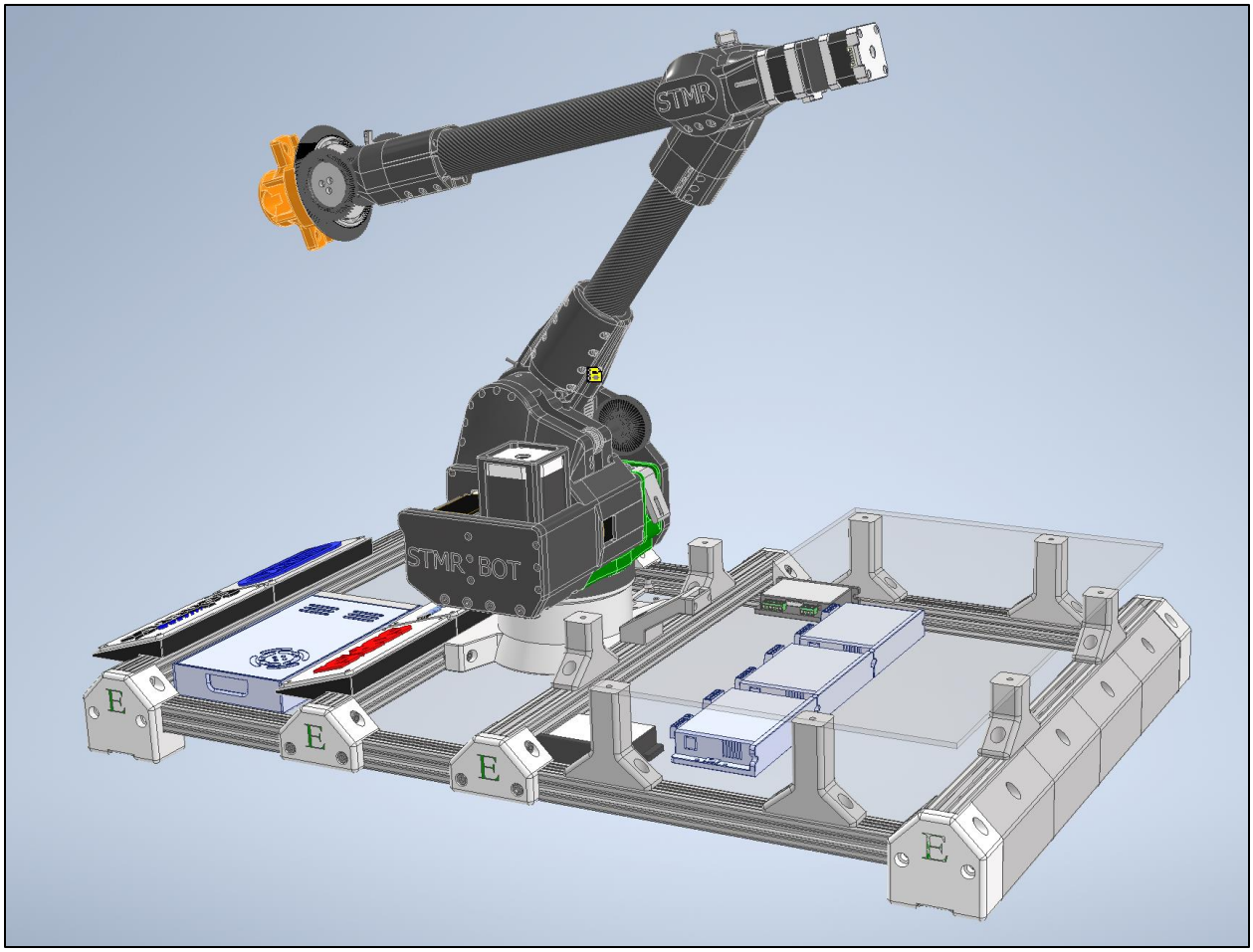

Figure.1: Completed CAD Model

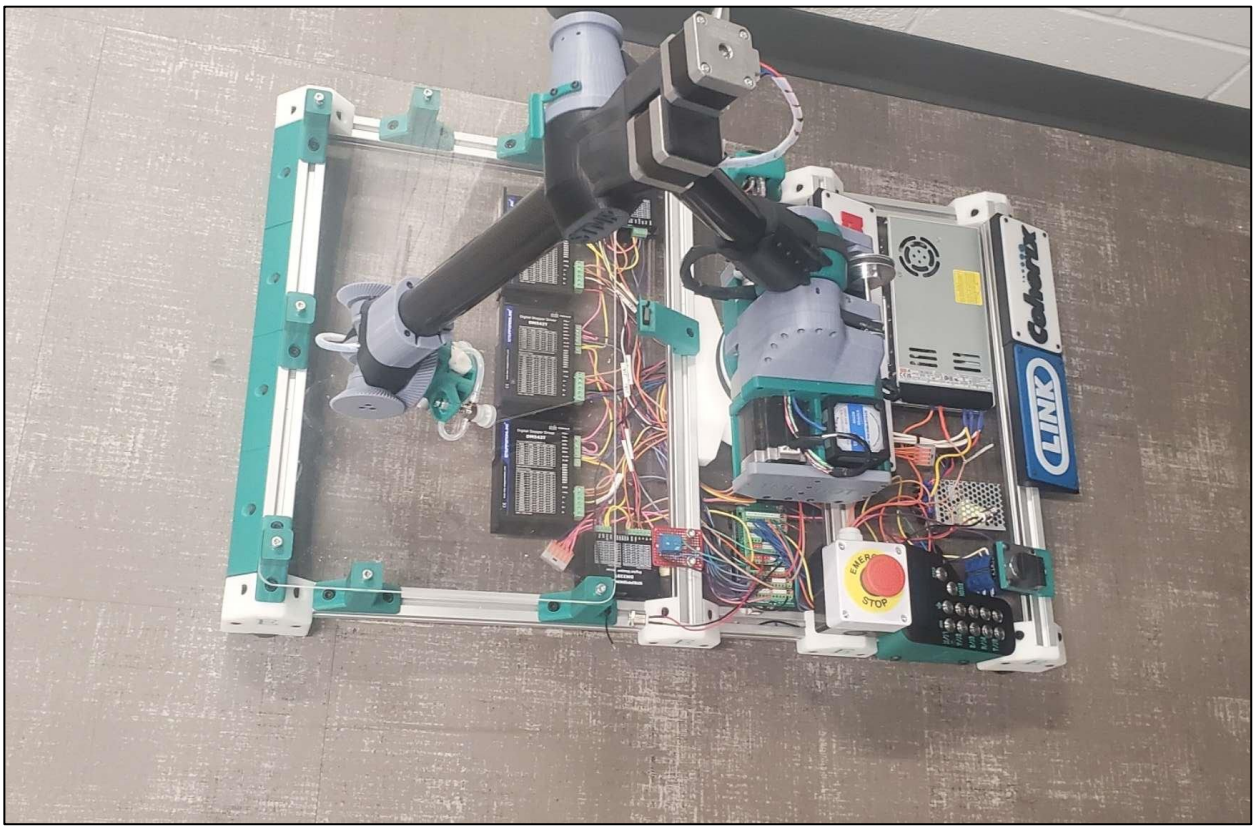

Figure.2: Completed assembly.

Robot Control Hardware

5 bipolar stepper motors, 5 digital stepper drivers, and 5 end stop microswitches are used to control the robots 5 DOF. An Arduino MEGA 2560 microcontroller is used for processing digital I/O and sending PWM signals to the stepper drivers for motion control. A rotary switch is used to select the operating mode of the robot (OFF, Homing, BCP, Record (TCP) and Execute), and a control panel with momentary switches is used for jogging the robot. The robot's end effector is a dual suction cup paired with a motor, air pump, and relay powered by a 5VDC onboard PSU. An onboard 24VDC PSU powers the stepper drivers/motors. All digital signals operate on 5V logic. See **Figure.3** below.

Mechanical Design

The robot is constructed from custom designed and 3D printed ABS plastic parts. The frame is constructed from 1” aluminum extrusion. Carbon fiber tubes were used for its lightweight stiffness, and the ability to route wires, drive belts, and drive shafts internally through their centers. 10 pulleys, 5 timing pulleys, 7 bevel gears, and a planetary gearset were used for obtaining proper torque and power at each joint based on payload requirements. Proper belt tension for each belt is achieved by adjustable motor mounts, or eccentric mounted idler bearings. A separate report details timing belt selection and the factor of safety for each belt drive. Design highlights for each joint are shown below.

Wrist Joint

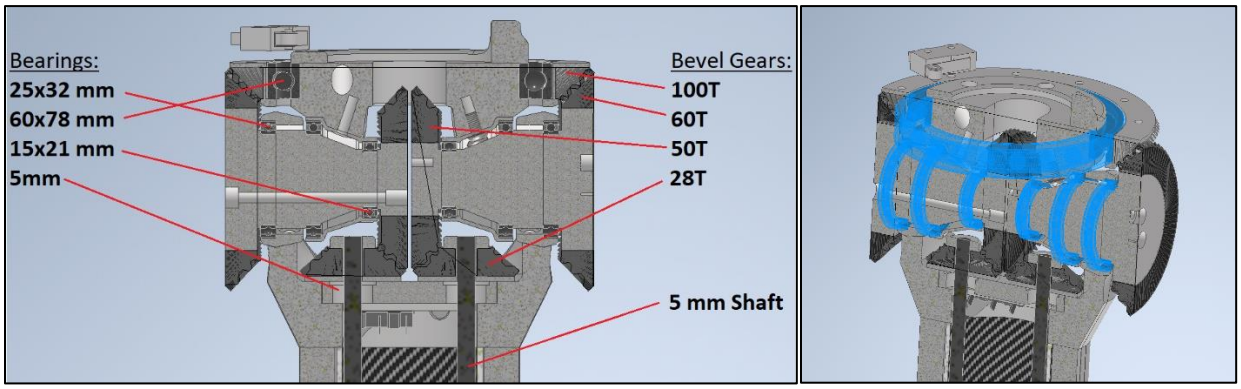

Figure.4: Wrist joint section views. Wrist joint provides 2 degrees of freedom using input from 5mm shafts/NEMA17 stepper motors. Bearings highlighted in blue (right).

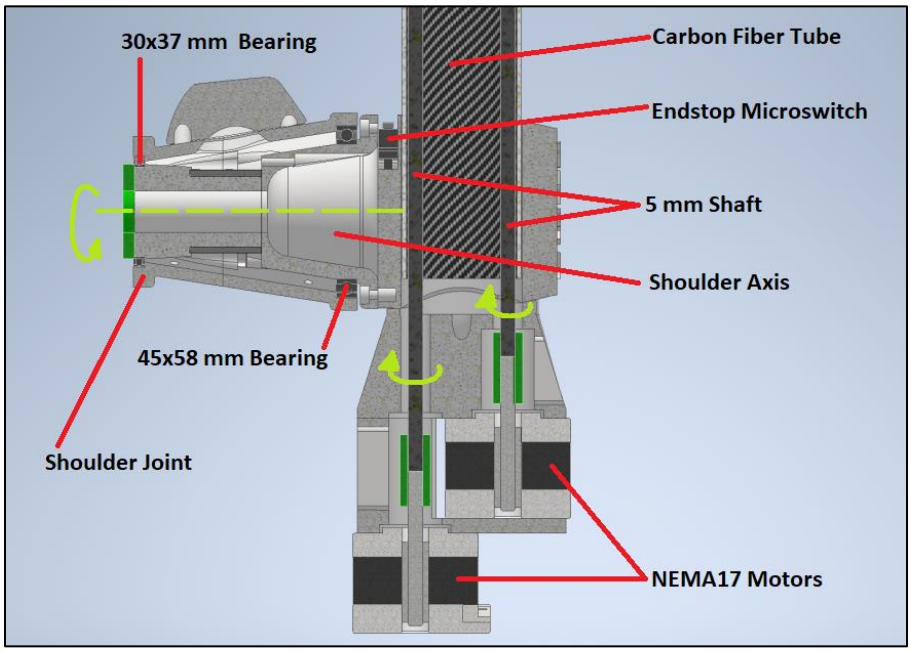

Figure.5: Elbow joint and wrist drive section view. Wrist drive motors located at elbow joint to minimize inertial mass at extremities of the robot and to counterbalance payload.

Elbow Joint

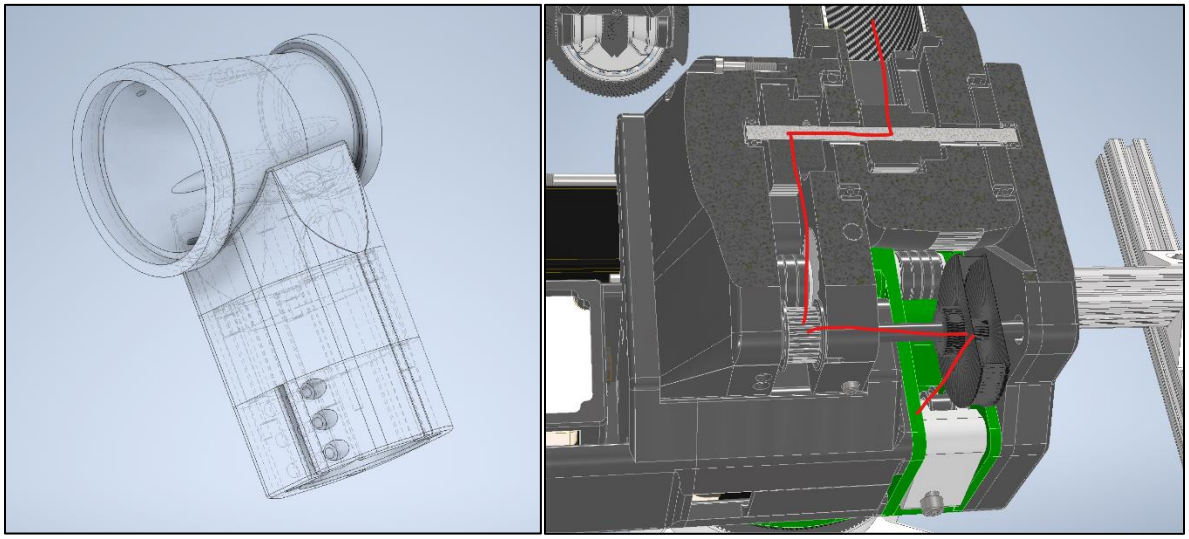

Figure.6: Assembled Elbow Joint constructed from 6X 3D-printed parts (left), Drivetrain path for elbow joint shown in red (right)

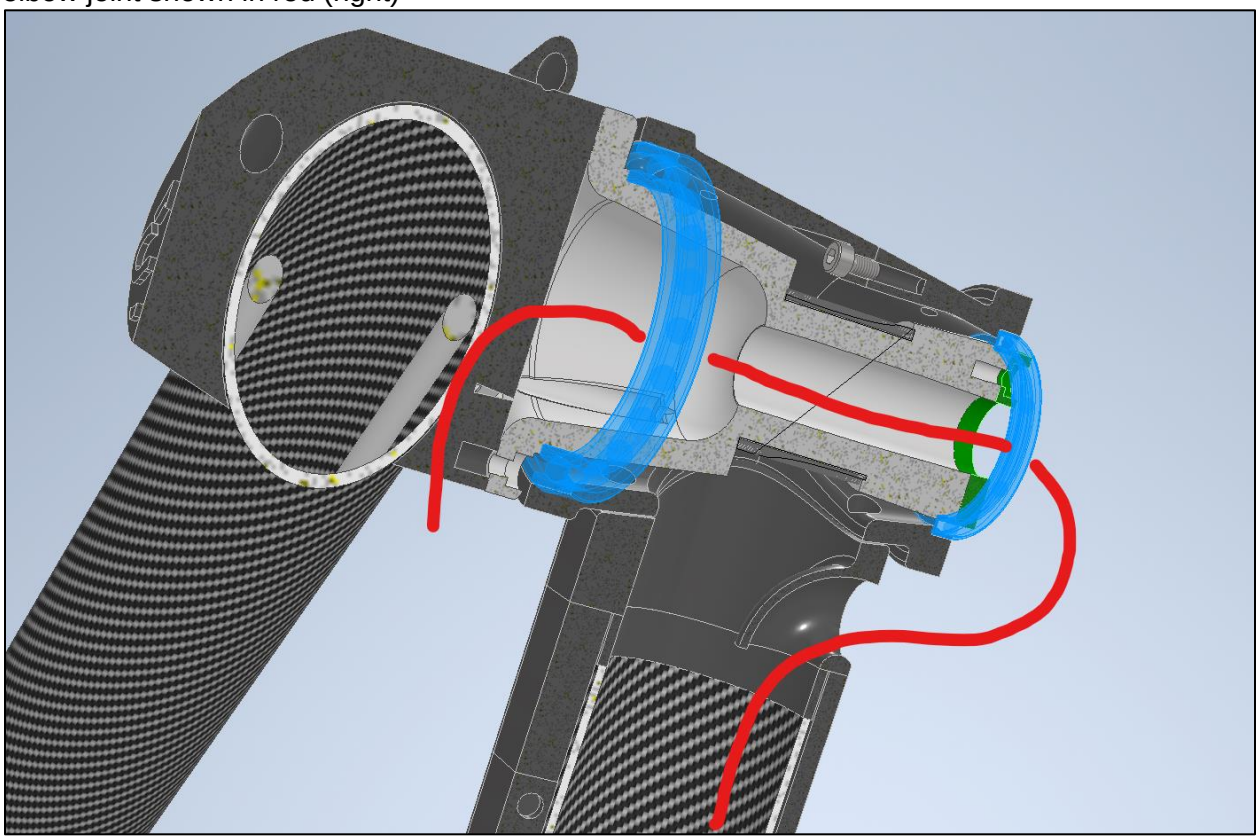

Figure.7: Bearings (blue) and wire routing (red) for elbow joint

Base Joint

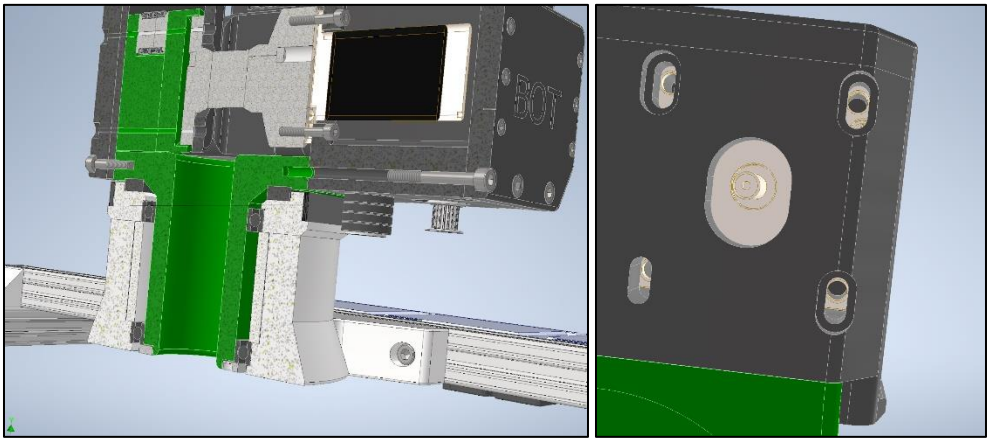

Figure.8: Section view of the base axis highlights a path for routing the main wiring harness (left). Motor mount slots allow for proper belt tensioning (right).

Kinematics and Programming

Forward kinematic analysis was completed analytically using MATLAB. A kinematic diagram was created (Figure.9) using the Denavit-Hartenberg criteria. $\Theta_0, \Theta_1, \Theta_2, \Theta_3, \Theta_4$ represent joint angles, and X_n, Y_n, Z_n represent coordinate system for the **n**th joint. Based on this diagram, rotation matrices and displacement vectors (Figure.10) were created to give the relative orientation and displacement of the **n** and **n+1** coordinate system. These individual matrices were used to generate a global 4x4 homogeneous transformation matrix (Figure.11), which outputs the orientation and displacement of the end-effector frame (X_5, Y_5, Z_5) relative to the base coordinate frame (X_0, Y_0, Z_0), given the user input of link dimensions and joint angles. This code will be ported into our C++ program and called as a function to calculate the location of our suction cup end effector.

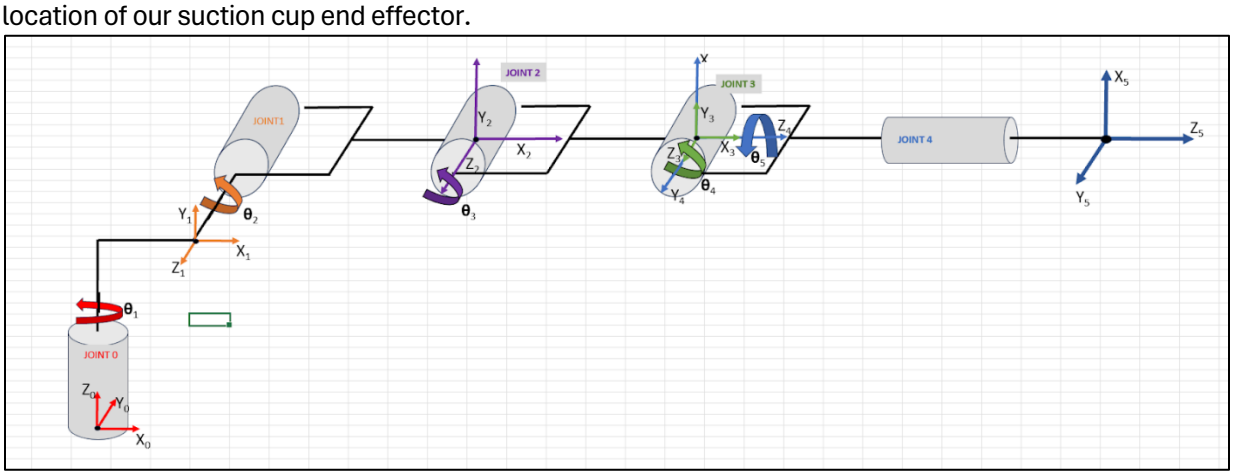

Figure.9: Kinematic Diagram from which transformations are derived (Joint 0 at robot base)

```
% Rotation Matrices

R01 = [1, 0, 0; 0, 0, -1; 0, 1, 0]*[cos(a), 0, sin(a); 0, 1, 0; -sin(a), 0, cos(a)];

R12 = [1, 0, 0; 0, 1, 0; 0, 0, 1]*[cos(b), -sin(b), 0; sin(b), cos(b), 0; 0, 0, 1];

R23 = [1, 0, 0; 0, 1, 0; 0, 0, 1]*[cos(c), -sin(c), 0; sin(c), cos(c), 0; 0, 0, 1];

R34 = [0, 0, 1; 1, 0, 0; 0, 1, 0]*[cos(d), 0, sin(d); 0, 1, 0; -sin(d), 0, cos(d)];

R45 = [1, 0, 0; 0, 1, 0; 0, 0, 1]*[cos(e), -sin(e), 0; sin(e), cos(e), 0; 0, 0, 1];

R05 = R01*R12*R23*R34*R45;
```

```
d01 = [a2*cos(a); a2*sin(a); a1];

d12 = [a4*cos(b); a4*sin(b); -a3];

d23 = [a5*cos(c); a5*sin(c); 0];

d34 = [0; 0; 0];

d45 = [0; 0; a6];
```

Figure.10: Rotation matrices (left) and displacement vectors (right) in MATLAB

```
% Homogeneous Transformation Matrices

H01 = [R01(1,1), R01(1,2), R01(1,3), d01(1,1);
       R01(2,1), R01(2,2), R01(2,3), d01(2,1);
       R01(3,1), R01(3,2), R01(3,3), d01(3,1);
          0         0         0          1   ];

H12 = [R12(1,1), R12(1,2), R12(1,3), d12(1,1);
       R12(2,1), R12(2,2), R12(2,3), d12(2,1);
       R12(3,1), R12(3,2), R12(3,3), d12(3,1);
          0         0         0          1   ];

H23 = [R23(1,1), R23(1,2), R23(1,3), d23(1,1);
       R23(2,1), R23(2,2), R23(2,3), d23(2,1);
       R23(3,1), R23(3,2), R23(3,3), d23(3,1);
          0         0         0          1   ];

H34 = [R34(1,1), R34(1,2), R34(1,3), d34(1,1);
       R34(2,1), R34(2,2), R34(2,3), d34(2,1);
       R34(3,1), R34(3,2), R34(3,3), d34(3,1);
          0         0         0          1   ];

H45 = [R45(1,1), R45(1,2), R45(1,3), d45(1,1);
       R45(2,1), R45(2,2), R45(2,3), d45(2,1);
       R45(3,1), R45(3,2), R45(3,3), d45(3,1);
          0         0         0          1   ];

% Solution, Forward Kinematics

H05 = H01*H12*H23*H34*H45
```

```
H05 =

   -0.7068         0    0.7074    0.6013
         0   -1.0000         0         0
    0.7074         0    0.7068    1.0008
         0         0         0    1.0000
```

Figure.11: Individual and global transformation matrices are generated from rotation matrices and displacement vectors (left), sample output (right). Red square shows global transform in X, Y, Z. Blue square gives global rotation (identity matrix indicates no rotation).

```
void inverse_kin_angles()    //give end effector target position, modifies motor angle target variables
{
  J0.targ_angle = atan(targ_Z / targ_X);

      float J4x = (targ_X / (cos(J0.targ_angle))) - (a6)*cos(targ_R);   //delete this eventually/combine formulas. easy to read for now
      float J4y = targ_Y - (a6)*sin(targ_R);                            //same as noted above
      float d = pow(pow(J4x, 2) + pow(J4y, 2), 1/2);                    //same as noted above

  //J2.targ_angle = acos((pow(d, 2) - C1)/(-2*(C4)));

  J0.targ_angle = atan(J4y/J4x) + acos((C3-(pow(d, 2)))/(-2*d*a4));

  //J3.targ_angle = targ_R - t1 - t2;

  //J4.targ_angle = targ_T;
 }
```

Figure.12: Inverse kinematics implemented in C++. Calculates motor angles for each joint to achieve an end-effector position/angle.

Photos

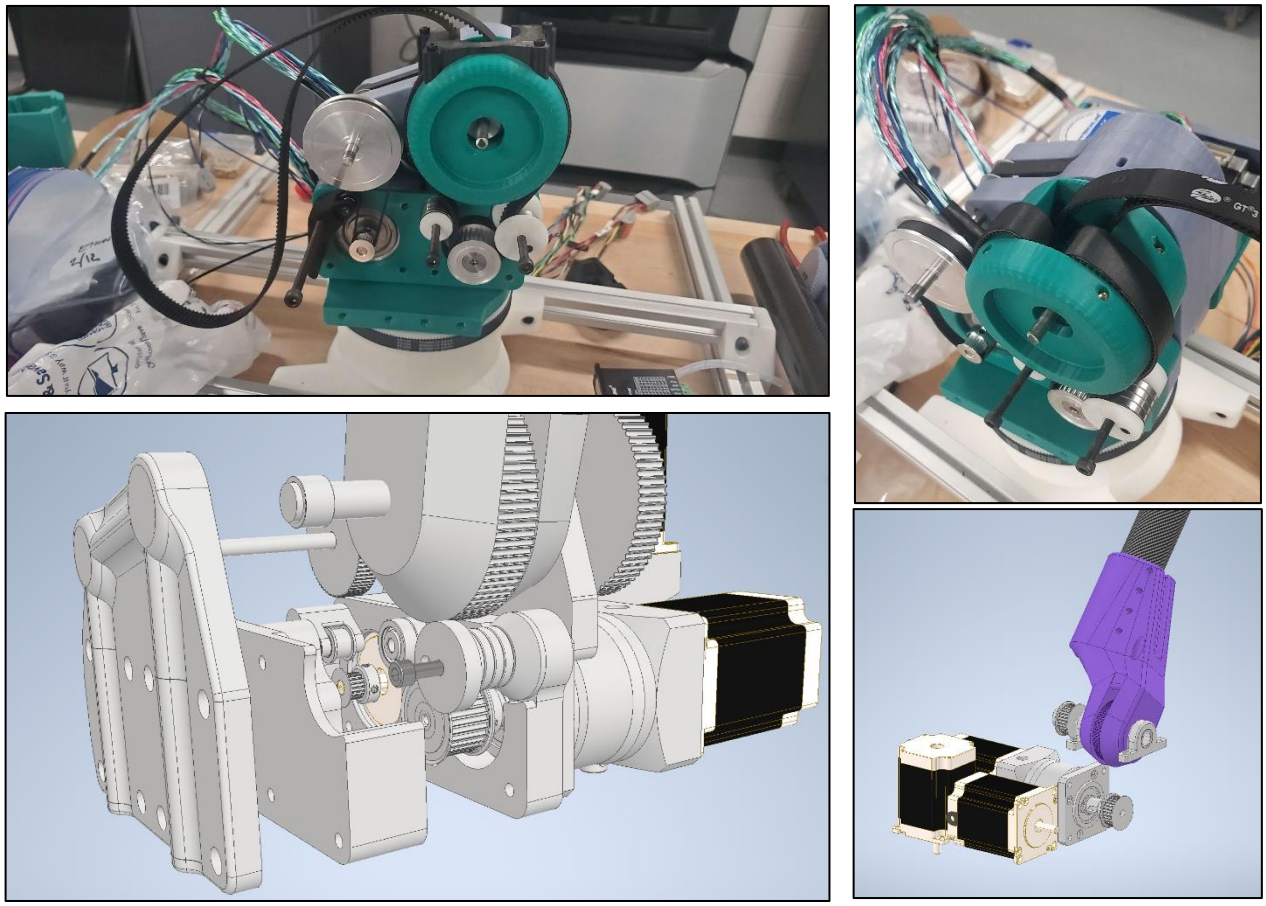

Figure.13: Initial design concepts for packaging motors and drivetrains (bottom left, bottom right). Drivetrain assembly for shoulder and elbow (top left, top right).

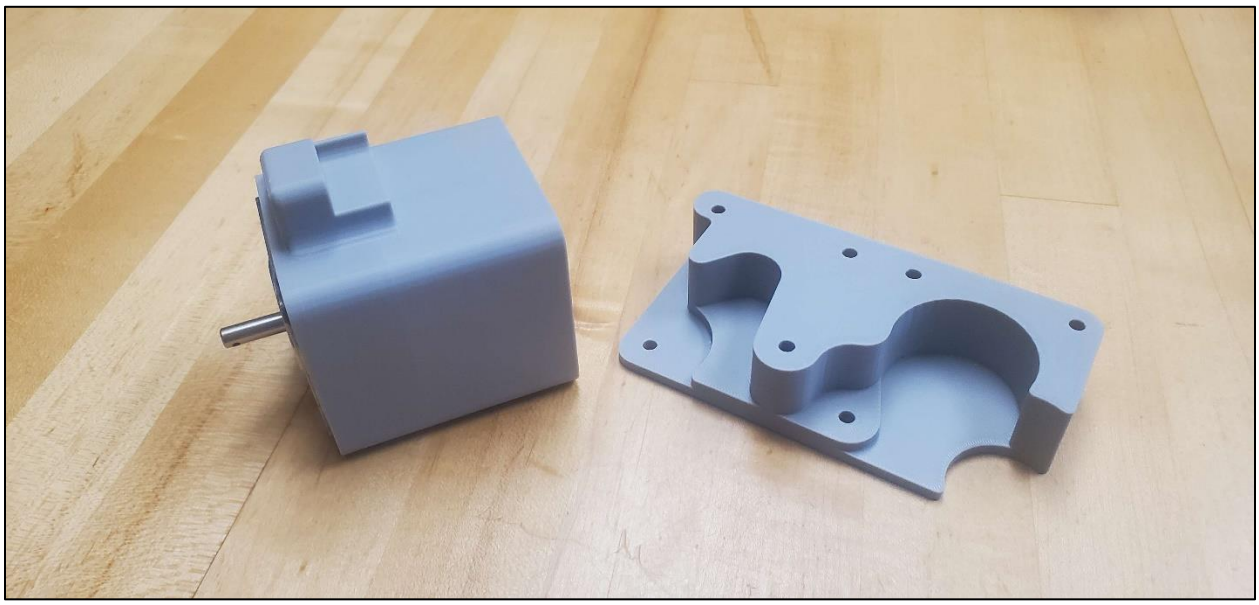

Figure.14: The first design iterations began printing on December 28, 2023

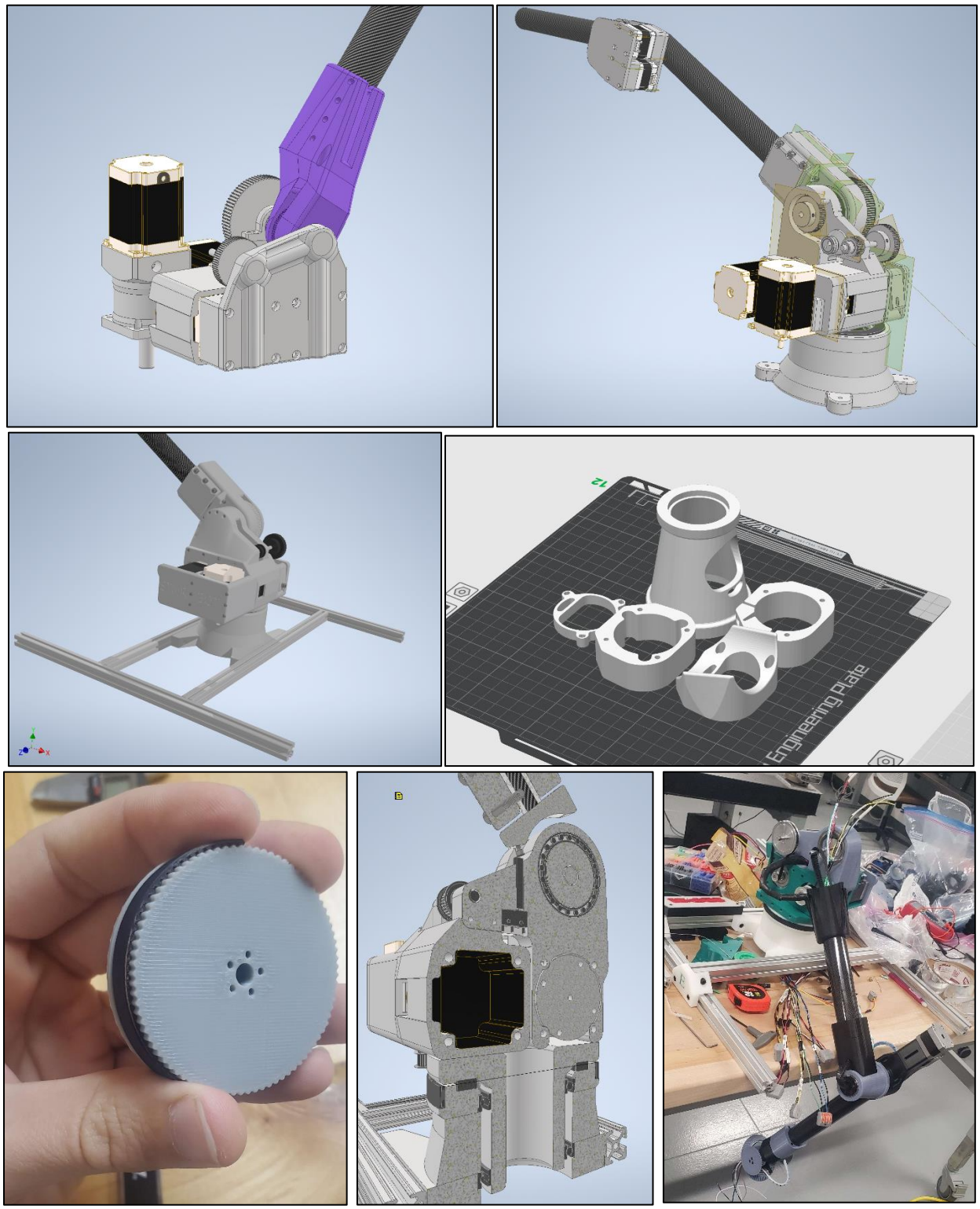

Figure.15: Design matures significantly in January-February 2024. 2nd and 3rd design iterations are printed, test fitted and assembled.

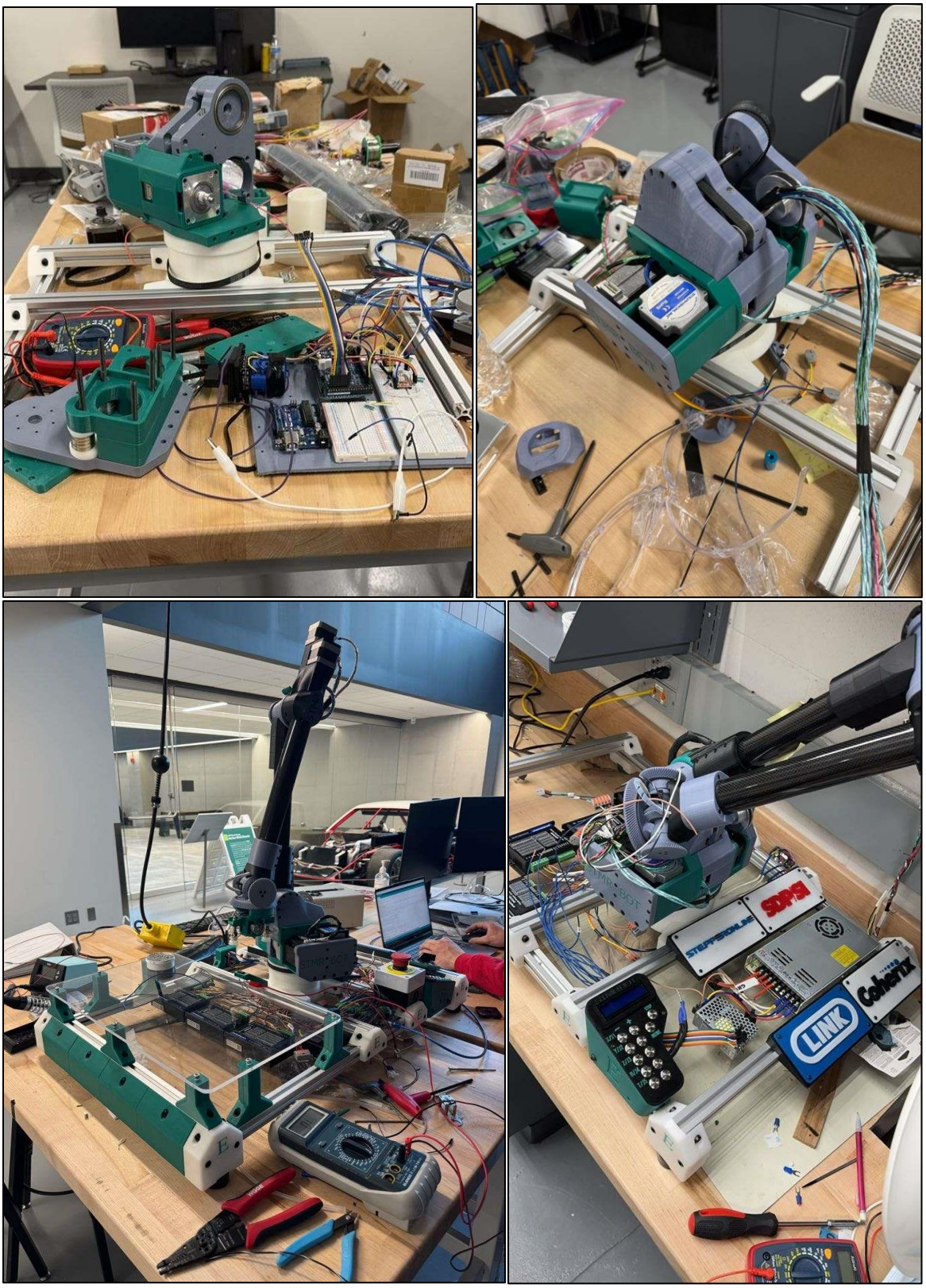

Figure.16: Partially assembled base and prototyping board for electronics setup (top left, top right, February 2024). Programming and running wiring harness (bottom left, bottom right, April 2024).

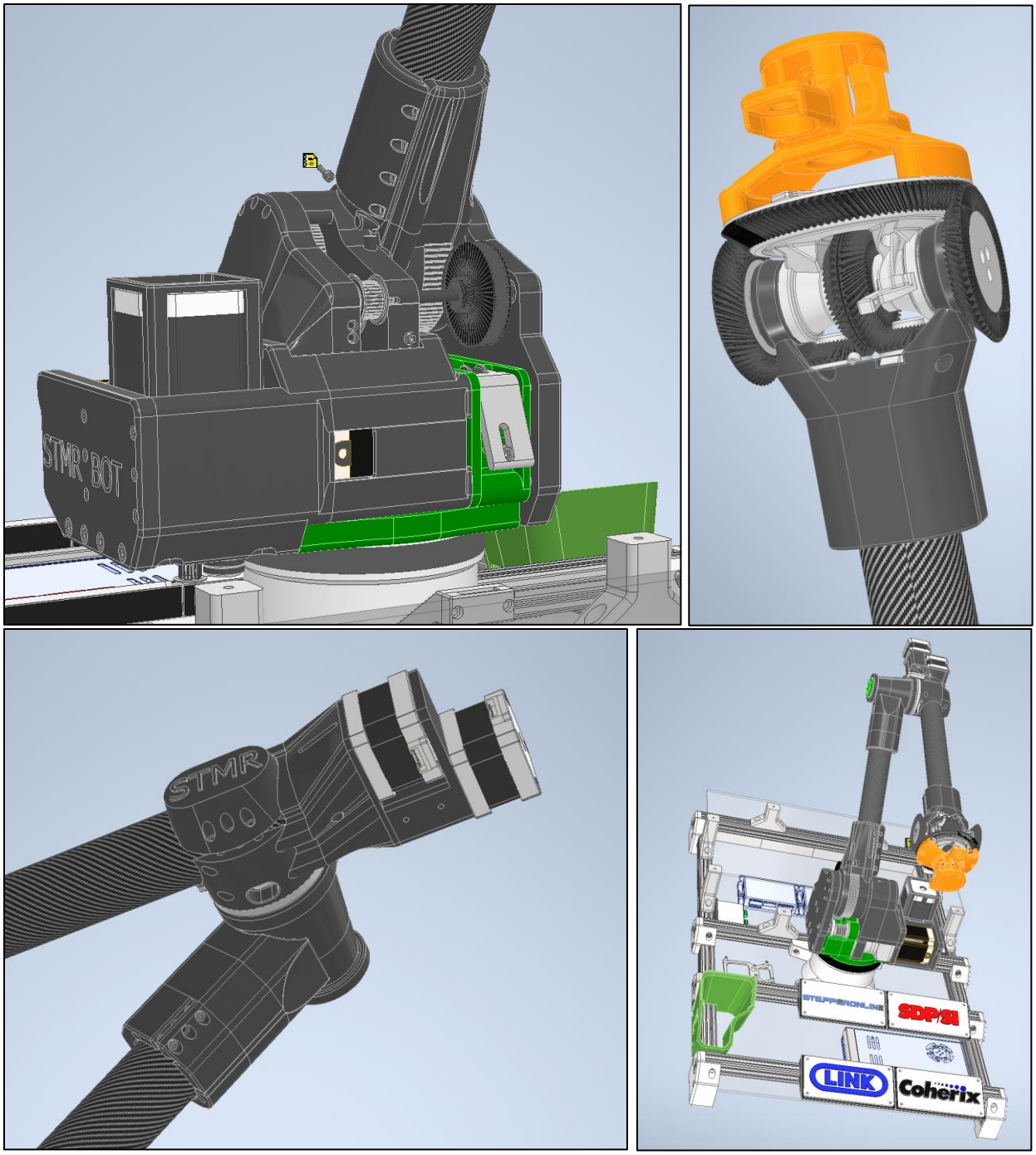

Figure.16: Final Design is completed and built by April 2024

Contributors and Sponsors

Mechanical design, programming, and kinematic calculations were completed by Scott Cann. Fabrication and assembly of the robot was completed by Scott Cann and Miles Burgan. Mechanical design of the end-effector was done by Ronnie Angelosanta. Reporting was completed by Scott Cann, Tim Sayers, Miles Burgan, and Ronnie Angelosanta.

We thank our generous sponsors for their contributions to this ambitious project – without their donations this would not have been possible: